

Tradexy — Complete Deployment Guide

Last updated: February 18, 2026 **Server:** xCloud Managed VPS (single server, 3 environments)

Database: Aiven PostgreSQL (dev), Supabase PostgreSQL (staging/prod)

Table of Contents

1. [Security Best Practices](#)
 2. [How It Works](#)
 3. [File Reference — What Each File Does](#)
 4. [GitHub Setup \(One-Time\)](#)
 5. [Server Setup \(One-Time\)](#)
 6. [Deploying](#)
 7. [Verifying a Deployment](#)
 8. [Manual Deploy \(deploy.sh\)](#)
 9. [Disk Cleanup](#)
 10. [Troubleshooting](#)
-

1. Security Best Practices

- **Never commit `.env` files** — they contain APP_KEY, DB passwords, API keys. The `.gitignore` should exclude them.
- **Never expose your server IP** in committed files — use placeholders like `<YOUR_SERVER_IP>`.
- **Use GitHub Secrets** for all sensitive CI/CD values (SSH keys, tokens, server IPs).
- **Disable password SSH login** — use SSH key-only authentication:

```
# On your server, edit /etc/ssh/sshd_config:
PasswordAuthentication no
PermitRootLogin prohibit-password
# Then restart: sudo systemctl restart sshd
```

- **Restrict firewall** — only open the ports you need:

```
ufw default deny incoming
ufw default allow outgoing
ufw allow ssh
ufw allow 8080      # Dev
ufw allow 8081      # Staging
ufw allow 8082      # Production
ufw enable
```

- **Keep Docker images minimal** — the `.dockerignore` ensures secrets and unnecessary files stay out of the image.
- **Use separate APP_KEYS** per environment — never share the same key across dev/staging/prod.
- **Rotate GH_PAT periodically** — set an expiration when creating your GitHub Personal Access Token.

2. How It Works

You push code → GitHub Actions runs tests → Builds Docker image → Pushes to GHCR

→ SCPs compose+nginx config to server

→ SSHs into server → Pulls image → Restarts containers

Branch	GHCR Image Tag	Server Directory	Port	Compose File	Database
dev	:dev	/var/www/tradexy-dev	8080	docker-compose.dev.yml	Aiven
staging	:staging	/var/www/tradexy-staging	8081	docker-compose.staging.yml	Supabase
main	:main	/var/www/tradexy-prod	8082	docker-compose.prod.yml	Supabase

3. File Reference — What Each File Does

.dockerignore

Purpose: Tells Docker what to EXCLUDE when building the image. Without this, `COPY . .` in the Dockerfile would copy `.git` (several GB), `node_modules`, `vendor`, etc., making the image huge.

`.git`
`.github`
`.env / .env.*`
`!.env.example`
`node_modules`
`vendor`
`storage/logs/*`
`storage/framework/cache/*`
`storage/framework/sessions/*`
`storage/framework/views/*`
`tests`
`phpunit.xml`
`docker-compose*.yml`
`deploy.sh`
`deployment-guide.md`
`README.md`

`# Git history – not needed in production`
`# GitHub Actions – not needed in image`
`# Secrets – NEVER bake into image`
`# Exception: keep the example file`
`# Frontend deps – not needed in PHP image`
`# Rebuilt by composer install during build`
`# Runtime logs – not needed in image`
`# Runtime cache`
`# Runtime sessions`
`# Compiled views`
`# Test files – not needed in production`
`# Test config`
`# Compose files – deployed separately via SCP`
`# Deploy script`
`# This file`
`# Readme`

```
.gitignore / .dockerignore    # Config files
.vscode / .idea               # IDE files
```

docker/php/Dockerfile (Development — local only)

Purpose: Builds the PHP container for **local development**. Not used on the server.

```
FROM php:8.4-fpm

# Install PHP extensions needed for Laravel + PostgreSQL
RUN apt-get update && apt-get install -y \
    git curl zip unzip libpng-dev libonig-dev \
    libxml2-dev libzip-dev libpq-dev \
    && docker-php-ext-install \
        pdo pdo_pgsql mbstring zip exif pcntl

# Install Composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

WORKDIR /var/www
COPY . .

# Set permissions
RUN chown -R www-data:www-data /var/www \
    && chmod -R 775 storage bootstrap/cache

# Install ALL dependencies (including dev)
RUN composer install --no-interaction --prefer-dist

EXPOSE 9000
CMD ["php-fpm"]
```

docker/php/Dockerfile.prod (Production — used by CI/CD)

Purpose: Builds the optimized PHP container image that gets pushed to GHCR. This is the image used on the server for ALL environments (dev/staging/prod).

```
FROM php:8.4-fpm

# Install system dependencies + PHP extensions
RUN apt-get update && apt-get install -y \
    git curl zip unzip libpng-dev libonig-dev \
    libxml2-dev libzip-dev libpq-dev libfcgi-bin \
    && docker-php-ext-install \
        pdo pdo_pgsql mbstring zip exif pcntl \
    && apt-get clean && rm -rf /var/lib/apt/lists/* # Clean up to reduce size
```

```
# Health check script (for monitoring)
RUN curl -o /usr/local/bin/php-fpm-healthcheck \
    https://raw.githubusercontent.com/renatomefi/php-fpm-healthcheck/master/php-
fpm-healthcheck \
    && chmod +x /usr/local/bin/php-fpm-healthcheck
RUN echo "pm.status_path = /status" >> /usr/local/etc/php-fpm.d/zz-docker.conf

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

WORKDIR /var/www

# Layer caching: copy composer files first, install deps, then copy code
# This means deps are only re-installed if composer.json/lock changes
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader --no-interaction --prefer-dist
--no-scripts

# Copy application code (filtered by .dockerignore)
COPY . .

# Rebuild autoloader with full code
RUN composer dump-autoload --optimize

# Set permissions
RUN chown -R www-data:www-data /var/www \
    && chmod -R 775 storage bootstrap/cache

EXPOSE 9000
CMD ["php-fpm"]
```

Key differences from dev Dockerfile:

- `--no-dev` — excludes dev dependencies
- `--optimize-autoloader` — faster class loading
- Layer caching — composer install runs before code copy
- Health check — for container monitoring
- `apt-get clean` — smaller image

docker/nginx/default.conf

Purpose: Nginx web server configuration. Routes HTTP requests to PHP-FPM. This file is SCP'd to the server during deployment.

```
server {
    listen 80;
    index index.php index.html;
    root /var/www/public;           # Laravel's public directory

    location / {
        try_files $uri $uri/ /index.php?$query_string;    # Pretty URLs
    }
}
```

```

}

location ~ /\.php$ {
    fastcgi_pass app:9000;          # Forward PHP to the app container
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    include fastcgi_params;
}

location ~ /\.ht {
    deny all;                      # Block .htaccess files
}
}

```

`docker-compose.yml` (Base — local development only)

Purpose: For running the app locally on your machine with a local PostgreSQL. **NOT used on the server.**

```

services:
  app:
    build:
      context: .
      dockerfile: docker/php/Dockerfile    # Builds from source locally
    container_name: laravel_app
    volumes:
      - ./var/www                          # Live code mount for hot reload
    networks:
      - laravel

  nginx:
    image: nginx:alpine
    container_name: laravel_nginx
    ports:
      - "8000:80"                          # Access at localhost:8000
    volumes:
      - ./var/www
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - app
    networks:
      - laravel

  postgres:
    image: postgres:16
    container_name: laravel_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_DATABASE}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}

```

```

    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - laravel

networks:
  laravel:

volumes:
  pgdata:

```

`docker-compose.dev.yml` (Dev server)

Purpose: Runs the app on the server for the **dev** environment. Pulls pre-built image from GHCR. No local PostgreSQL (uses Aiven).

```

services:
  app:
    image: ghcr.io/codebykenth/trading-journal-v2:dev  # Pre-built image from
    GHCR
    container_name: tradexy_app_dev
    restart: unless-stopped
    working_dir: /var/www
    env_file: .env  # ← This loads your .env
    into the container
    volumes:
      - app_storage_dev:/var/www/storage  # Persist uploads/logs
    across deploys
    networks:
      - tradexy_dev

  nginx:
    image: nginx:alpine
    container_name: tradexy_nginx_dev
    restart: unless-stopped
    ports:
      - "8080:80"  # Access at
    http://<server-ip>:8080
    volumes:
      - app_storage_dev:/var/www/storage:ro
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app
    networks:
      - tradexy_dev

networks:
  tradexy_dev:

```

```
volumes:
  app_storage_dev:           # Named volume – persists
  data
```

`docker-compose.staging.yml` (Staging server)

Purpose: Same as dev but for **staging**. Port 8081. Uses Supabase DB.

```
services:
  app:
    image: ghcr.io/codebykenth/trading-journal-v2:staging
    container_name: tradexy_app_staging
    # ... same structure, port 8081
```

`docker-compose.prod.yml` (Production server)

Purpose: Same as dev but for **production**. Port 8082. Uses Supabase DB. Image tag is `:main`.

```
services:
  app:
    image: ghcr.io/codebykenth/trading-journal-v2:main
    container_name: tradexy_app_prod
    # ... same structure, port 8082
```

`.github/workflows/ci-cd.yml`

Purpose: The CI/CD pipeline. Triggered on every push to `dev`, `staging`, or `main`. Has 5 jobs:

Job	What it does
1. Test	Installs PHP 8.4, runs <code>composer install</code> , creates test DB, runs <code>php artisan test</code>
2. Build & Push	Builds Docker image using <code>Dockerfile.prod</code> , pushes to <code>ghcr.io/codebykenth/trading-journal-v2:<branch></code>
3. Deploy Dev	SCPs <code>compose+nginx</code> to server, pulls image, restarts containers, runs migrations, clears cache
4. Deploy Staging	Same but caches <code>config/routes/views</code> (optimized for performance)
5. Deploy Prod	Same as staging

GitHub Secrets required:

Secret	Value
SERVER_HOST	Your VPS IP address (e.g. <YOUR_SERVER_IP>)
SERVER_USERNAME	SSH username (e.g. root)
SERVER_SSH_KEY	Your private SSH key (full content)
SERVER_SSH_PASSPHRASE	SSH key passphrase (if any)
GH_PAT	GitHub Personal Access Token with read:packages scope

deploy.sh (Manual deploy script)

Purpose: Optional script for manually deploying from the server terminal instead of using CI/CD. Useful for emergency deploys or debugging.

 **Note:** This file still has placeholder values. Update line 56:

```
# Change this:
docker pull ghcr.io/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME:$IMAGE_TAG
# To this:
docker pull ghcr.io/codebykenth/trading-journal-v2:$IMAGE_TAG
```

Usage (on the server):

```
./deploy.sh dev
./deploy.sh staging
./deploy.sh production
```

4. GitHub Setup (One-Time)

Step 1: Set workflow permissions

- 1. Go to your repo → **Settings** → **Actions** → **General**
- 2. Scroll to **Workflow permissions**
- 3. Select **"Read and write permissions"**
- 4. Check **"Allow GitHub Actions to create and approve pull requests"**
- 5. Click **Save**

Step 2: Add repository secrets

- 1. Go to your repo → **Settings** → **Secrets and variables** → **Actions**
- 2. Click **New repository secret** for each:

Secret Name	Value
SERVER_HOST	<YOUR_SERVER_IP>
SERVER_USERNAME	root
SERVER_SSH_KEY	Paste full private key from ~/.ssh/<YOUR_KEY>
SERVER_SSH_PASSPHRASE	Your key passphrase
GH_PAT	GitHub Personal Access Token (create at github.com/settings/tokens)

Step 3: Create GitHub PAT (if not done)

1. Go to github.com/settings/tokens
2. Click **Generate new token (classic)**
3. Select scopes: `read:packages`, `write:packages`
4. Copy the token → add as `GH_PAT` secret

5. Server Setup (One-Time)

SSH into your server:

```
ssh -i ~/.ssh/<YOUR_KEY> root@<YOUR_SERVER_IP>
```

Step 1: Create directories for all 3 environments

```
sudo mkdir -p /var/www/tradexy-dev/docker/nginx
sudo mkdir -p /var/www/tradexy-staging/docker/nginx
sudo mkdir -p /var/www/tradexy-prod/docker/nginx
```

Step 2: Create `.env` for dev

```
nano /var/www/tradexy-dev/.env
```

Paste your dev environment config:

```
APP_NAME=Tradexy
APP_ENV=local
APP_KEY=base64:PASTE_YOUR_KEY_HERE
APP_DEBUG=true
APP_URL=http://<YOUR_SERVER_IP>:8080

LOG_CHANNEL=stack
LOG_LEVEL=debug
```

```
DB_CONNECTION=pgsql
DB_HOST=your-aiven-host.aivencloud.com
DB_PORT=12345
DB_DATABASE=your_db_name
DB_USERNAME=your_username
DB_PASSWORD=your_password

# Add all other env vars (mail, social auth, API keys, etc.)
```

Generate APP_KEY locally:

```
php artisan key:generate --show
```

Copy the `base64:...` output.

Step 3: Create `.env` for staging

```
nano /var/www/tradexy-staging/.env
```

```
APP_NAME=Tradexy
APP_ENV=staging
APP_KEY=base64:PASTE_YOUR_KEY_HERE
APP_DEBUG=false
APP_URL=http://139.180.137.161:8081

DB_CONNECTION=pgsql
DB_HOST=your-supabase-host.supabase.co
DB_PORT=5432
DB_DATABASE=postgres
DB_USERNAME=your_username
DB_PASSWORD=your_password

# ... rest of staging config
```

Step 4: Create `.env` for production

```
nano /var/www/tradexy-prod/.env
```

```
APP_NAME=Tradexy
APP_ENV=production
APP_KEY=base64:PASTE_YOUR_KEY_HERE
```

```
APP_DEBUG=false
APP_URL=http://139.180.137.161:8082

DB_CONNECTION=pgsql
DB_HOST=your-supabase-host.supabase.co
DB_PORT=5432
DB_DATABASE=postgres
DB_USERNAME=your_username
DB_PASSWORD=your_password

# ... rest of production config
```

Step 5: Set permissions

```
sudo chown -R $USER:$USER /var/www/tradexy-dev
sudo chown -R $USER:$USER /var/www/tradexy-staging
sudo chown -R $USER:$USER /var/www/tradexy-prod
```

Step 6: Open firewall ports

```
ufw allow 8080
ufw allow 8081
ufw allow 8082
```

Step 7: Verify directory structure

After the first CI/CD deploy, each directory should look like:

```
/var/www/tradexy-dev/
├── .env                ← You created this (Step 2)
├── docker-compose.dev.yml ← Auto-synced by CI/CD
├── docker/
│   └── nginx/
│       └── default.conf ← Auto-synced by CI/CD
```

6. Deploying

Deploy to Dev

```
git checkout dev
# make your changes, commit...
git push origin dev
```

Deploy to Staging

```
git checkout staging
git merge dev
git push origin staging
```

Deploy to Production

```
git checkout main
git merge staging
git push origin main
```

Monitor

Go to GitHub repo → **Actions** tab → watch the pipeline.

Access in browser

Environment	URL
Dev	http://139.180.137.161:8080
Staging	http://139.180.137.161:8081
Production	http://139.180.137.161:8082

7. Verifying a Deployment

SSH into the server and run:

```
# Dev
cd /var/www/tradexy-dev
docker-compose -f docker-compose.dev.yml ps
docker-compose -f docker-compose.dev.yml exec app php artisan tinker --
execute="echo config('app.key');"
docker-compose -f docker-compose.dev.yml logs app
docker-compose -f docker-compose.dev.yml logs nginx

# Staging
cd /var/www/tradexy-staging
docker-compose -f docker-compose.staging.yml ps

# Production
cd /var/www/tradexy-prod
docker-compose -f docker-compose.prod.yml ps
```

8. Manual Deploy (deploy.sh)

If CI/CD fails or you need to deploy manually from the server:

```
# First, copy deploy.sh to the server (from your local machine)
scp -i ~/.ssh/xcloud_new deploy.sh root@139.180.137.161:/var/www/

# SSH in
ssh -i ~/.ssh/xcloud_new root@139.180.137.161

# Make it executable
chmod +x /var/www/deploy.sh

# Deploy
cd /var/www
./deploy.sh dev
./deploy.sh staging
./deploy.sh production
```

9. Disk Cleanup

Remove old git clone data (one-time)

```
# Check what's using disk space
du -sh /var/www/tradexy-dev/*
du -sh /var/www/tradexy-staging/*
du -sh /var/www/tradexy-prod/*

# Remove old source code (keep .env and docker configs!)
cd /var/www/tradexy-dev
rm -rf .git vendor node_modules app config database public resources routes
storage tests bootstrap artisan composer.* package.* webpack.* vite.*
```

Docker cleanup

```
docker system df          # Check Docker disk usage
docker image prune -af     # Remove unused images
docker volume prune -f     # Remove unused volumes
docker system prune -af    # Remove everything unused
```

10. Troubleshooting

Problem	Solution
---------	----------

Problem	Solution
APP_KEY error	Check <code>.env</code> exists in the server dir and has <code>APP_KEY=base64:...</code>
Bad credentials on build	Repo → Settings → Actions → General → Set workflow permissions to "Read and write"
unknown shorthand flag: 'f'	Server has Docker Compose V1 — use <code>docker-compose</code> (with hyphen)
Container won't start	<code>docker-compose -f docker-compose.<env>.yaml logs app</code>
Can't connect to DB	Verify DB credentials in <code>.env</code> . Check Aiven/Supabase allows your server IP
Nginx 502 Bad Gateway	App container crashed — check app logs
Permission errors	<code>docker-compose -f docker-compose.<env>.yaml exec app chown -R www-data:www-data /var/www/storage /var/www/bootstrap/cache</code>
CI/CD fails on SCP	Ensure <code>SERVER_SSH_KEY</code> secret has the correct full private key
Image not found	Check GHCR: https://github.com/codebykenth/trading-journal-v2/pkgs/container/trading-journal-v2